# Mysql架构与执行流程

## 一条SQL语句是如何执行的

### mysql客户端和服务器通讯

mysql单个客户端跟服务器的通讯在任何一个时刻，都是不能同时发生的，要么客户端向服务器发送数据，要么服务器向客户端发送数据！我们知道，当一个查询语句提交给服务器后，你是不能做其他事情的，只能等待结果，这也是为什么要进行sql语句优化，影响性能

### 连接状态查询

对于mysql的连接，任何时刻都会有一个状态，这个状态表示mysql当前正在做什么！！但是在查询的整个生命周期中，状态会发生变化，有以下这些状态

- sleep：线程正在等待客户端发送新的请求；
- query：线程正在执行查询或者正在将结果发送给客户端；
- locked：在mysql服务器层，该线程正在等待表锁。在存储引擎级别实现的锁，例如InnoDB的行锁，并不会体现在线程状态中。对于MyISAM来说这是一个比较典型的状态。
- analyzing and statistics：线程正在收集存储引擎的统计信息，并生成查询的执行计划；
- copying to tmp table：线程在执行查询，并且将其结果集复制到一个临时表中，这种状态一般要么是做group by操作，要么是文件排序操作，或者union操作。如果这个状态后面还有on disk标记，那表示mysql正在将一个内存临时表放到磁盘上。
- sorting Result：线程正在对结果集进行排序。
- sending data：线程可能在多个状态间传送数据，或者在生成结果集，或者在想客户端返回数据。

我们可以通过

```
show full processlist
```

咕泡出品

查询！

# 查询缓存（mysql8已废除）

该步骤只有查询语句才会有，它是以查询语句，并且大小写敏感的的哈希值查找实现的，只要多一个字符，哪怕是空格，或者大小写不完全一致，就不会走到查询缓存！并且包含不确定的函数也不会缓存，比如now()

如果查询命中缓存，后续的流程都不会走，但是因为不好用，因为是根据sql语句完成相同才会有用，所以使用场景有局限性，mysql8已经废除此功能

# 解析器

## 词法解析

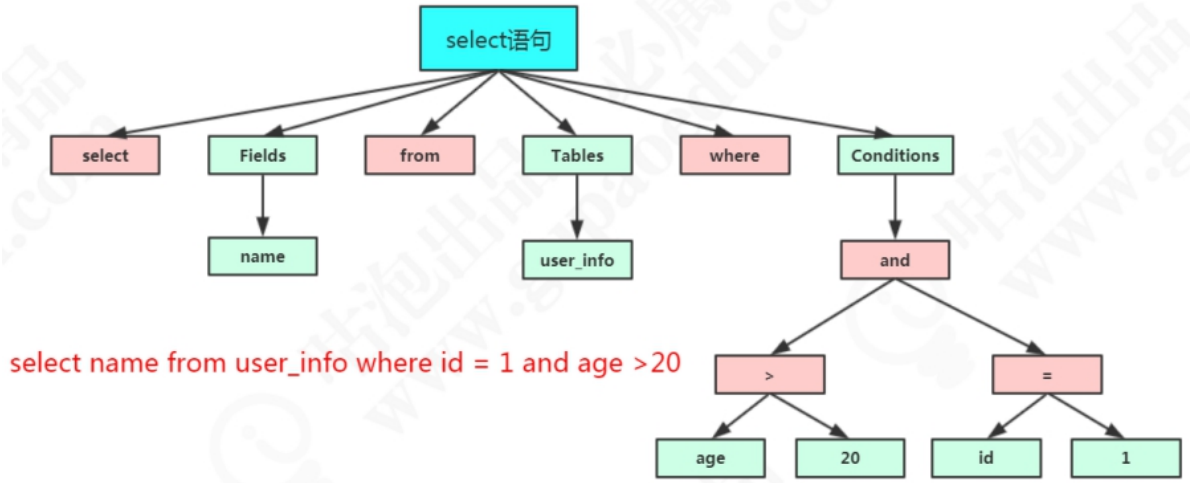
词法解析就是把一个完整的SQL语句打碎成一个个的单词。

```
select name from user where id = 1;
```

它会打碎成8个符号，每个符号是什么类型

## 语法解析

进行语法解析，判断输入的这个 SQL 语句是否满足 MySQL 语法。然后生成下面这样一颗语法树：




它会检查红色的关键词语法，如果语法不对，会报错，但是不会去检查相关表名，列名是否正确！

## 预处理器

解析器不会去检查查询的表名、列名是否正确，是否有表的权限等，所以我们需要用到处理器去进行进一步检查解析树是否合法，它会检查生成的解析树，解决解析器无法解析的语义。比如，它会检查表和列名是否存在，检查名字和别名，保证没有歧义。

## 优化器

当上述步骤完成，则sql语句认为是合法并且是可执行的，并且可以有很多的执行方式，并且返回相同的结果，优化器会把一条sql语句转化成执行计划，并且找到其中最好的执行计划！

MySQL的优化器能处理哪些优化类型呢？

举两个简单的例子：

1、当我们对多张表进行关联查询的时候，以哪个表的数据作为基准表。

2、有多个索引可以使用的时候，选择哪个索引。

实际上，对于每一种数据库来说，优化器的模块都是必不可少的，他们通过复杂的算法实现尽可能优化查询效率的目标。

到此我们就得到了一个执行计划，是不是要交给执行器

## 执行器

利用存储引擎提供的相应的API来完成操作。并且把数据返回给客户端

## 存储引擎

存储引擎就是我们的数据真正存放的地方，在MySQL里面支持不同的存储引擎。现默认的存储引擎为InnoDB

这个大家可以为我们文件的格式，虽然内容是一样的，但是存储的文件方式可以有很多，就像我们文本数据信息，我们可以存储.txt，也可以存储为.sql，也可以存储为.js，也可以是md! 只是存储方式不一样，在格式，大小，速度方面有不同!

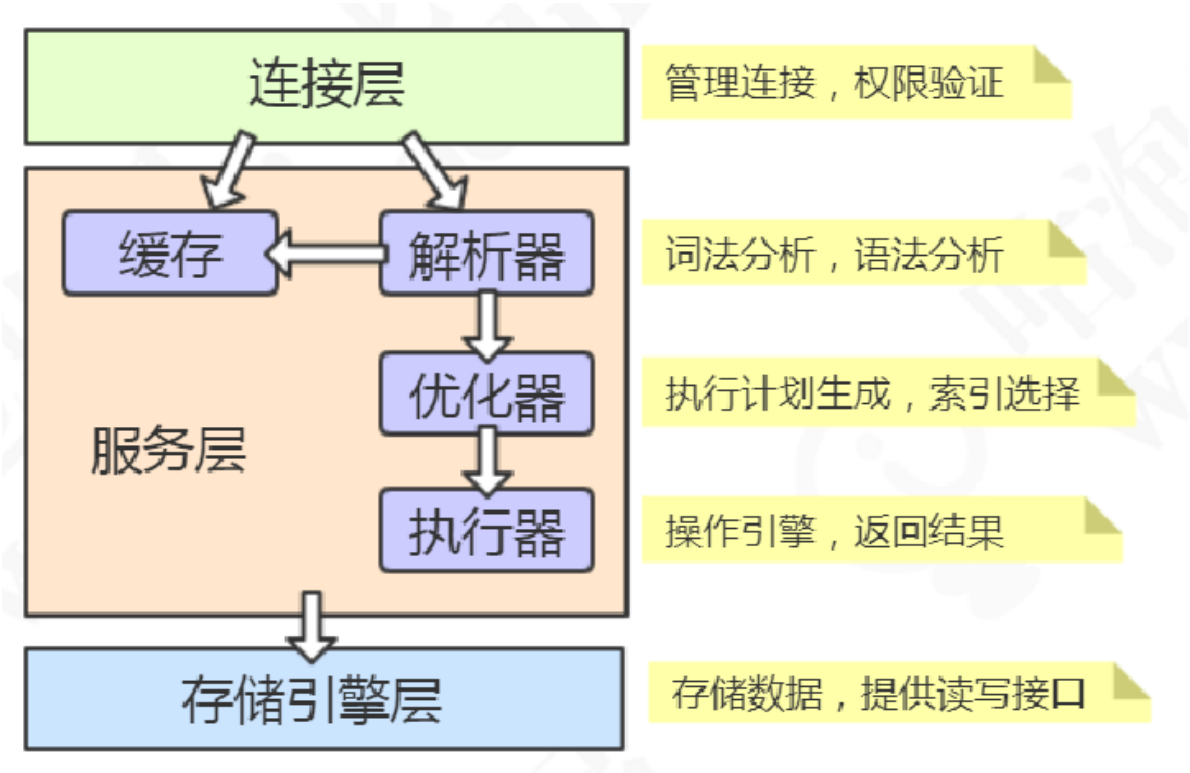
咕泡出品

```sql
CREATE TABLE `student` (
  `student_no` int(16) NOT NULL COMMENT '学号',
  `student_name` varchar(64) DEFAULT NULL COMMENT '姓名',
  `student_age` int(4) DEFAULT NULL COMMENT '年龄',
  `student_addr` varchar(128) DEFAULT NULL COMMENT '地址',
  PRIMARY KEY (`student_no`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

SELECT ENGINE from  information_schema.tables where
table_name='student' --查询某个表的存储引擎
```

官方地址：[https://dev.mysql.com/doc/refman/8.0/en/storage-engines.html](https://dev.mysql.com/doc/refman/8.0/en/storage-engines.html)

## Mysql体系架构总结




## Buffer pool

我们都知道，Mysql 数据最终是要放到磁盘的，但是，如果我们每次增删改都直接从磁盘操作，讲会很慢很慢，所以mysql也会有一个机制，会把磁盘的数据读到内存中进行操作，然后回写到磁盘！那么这个内存区域就是我们讲的buffer
pool，Buffer Pool默认大小是128M（134217728字节），可以调整。

~~我们可以通过语句~~

```
SHOW STATUS LIKE '%innodb_buffer_pool%';
```

~~进行查询，相关参数参考~~[https://dev.mysql.com/doc/refman/8.0/en/server-system-variables.html](https://dev.mysql.com/doc/refman/8.0/en/server-system-variables.html)

读取的时候，不是把磁盘所有的数据读取到buffer
pool，也不是要多少读多少，InnoDB设定了一个存储引擎从磁盘读取数据到内存的最小的单位，叫做页。操作系统也有页的概念。操作系统的页大小一般是4K，而在InnoDB里面，这个最小的单位默认是16KB大小。后续会讲。

后续的读取跟写入，都优先操作buffer pool，而不是直接写到磁盘，内存的数据页和磁盘数据不一致的时候，我们把它叫做脏页

现在我知道了一条sql语句是以上的流程，并且不是每次都是去磁盘拿取数据的，而是会有一个内存模块叫做Buffer pool!!提升了速度

那这块具体怎么拿的，也是需要知道?

bufferpool跟磁盘交互的数据单元 叫 page页；每次交互数据都是一页一页的，不会是一条一条的！！，这个最小的单位默认是16KB大小；具体page怎么存数据的，后续会讲。

什么叫做按page页交互，比如我现在一条select的语句 select * from table where id=5;

咕泡出品

它的步骤为：

1.通过语法词法解析，优化器、执行器。调用innoDB的存储引擎

2.从buffer里面去拿。。。没有，去读取磁盘数据。

3.磁盘就不是读一条数据，而是根据page去读，读16KB的数据，所以可能会把1.2.3.4.5.6.7的在一页里面的数据全部放到buffer

4.下次我去查询id=6的数据，就可以不与磁盘交互

# 修改内存已有数据逻辑

比如update id=5的数据

我们发现在buffer里面已经有这条数据了。只需要更改内存的数据页即可，而这个被更改的页也就叫做脏页（因为跟我磁盘的数据不一致了）。

既然不一致了，那我们还需要把内存更改过的数据同步到磁盘，那么这个动作就叫做刷脏。

那么刷脏的时候怎么找到我的脏页？

这些脏页会在我们bufferpool里面有一个专门的脏页链表，叫做flush链表。这样我去找我的脏页只要去这个链表找就行，不需要在我内存的所有的页面里面去找了。

那么刷脏什么时候进行？

最简单的办法，我每次更改，更改内存的同时，我立即同步到磁盘！！但是磁盘慢得跟乌龟一样，会严重影响程序的性能，所以我们不立即同步给磁盘；我们有自己的一些刷脏模式

我们有专门的线程每隔一段时间就把脏页刷新到磁盘，不影响用户线程正常的请求操作。这些线程叫做Page Cleaner，可以配置线程数量。innodb-page-cleaners

刷脏的模式也有很多，比如

**基本模式** 根据redo产生的速度来决定

**脏页自适应** 根据Redo的产生速度和脏页的使用比例共同来决定刷脏速度。通过innodb_max_dirty_pages_pct_lwm和innodb_max_dirty_pages_pct来控制

**Redo自适应模式** 根据Redo的产生速度和Redo的使用比例共同来决定刷脏速度。通过innodb_adaptive_flushing和innodb_adaptive_flushing_lwm参数控制

空闲模式 等等

## Double Write机制

我们发现，操作数据都是一页一页的操作的，一页里面有很多很多的数据。那么假如，我一个页里面有些成功，有些失败，那怎么办！！

所以，为了保证页的数据一致性，又有了 Double Write的机制

1.在刷盘覆盖磁盘的时候，我们会将page内容写入到磁盘的另外一个地方，叫做doublewrite buffer。

2.然后再把内存里面的数据同步到磁盘

如果第一步失败，原来的数据没有覆盖，数据页是完整的

如果第二步失败，原来的数据不完整，但是doublewrite buffer是完整的，可以用doublewrite buffer里面的覆盖

**以上是我们去修改一条在bufferpool中有的数据，但是如果我改一条bufferpool里面没有的数据咋办？是不是需要从磁盘读出来？**

其实我们也不需要从磁盘里面读取出来，而是在bufferpool的内存空间里面又分了一块出来，叫做change buffer，可以通过参数innodb_change_buffer_max_xize来动态设置，这个参数为50的时候，标识change buffer 的大小最多只能占用 buffer pool 的50%。

咕泡出品

~~1.首先记录到redoLog中，防止机器意外关闭导致数据丢失，会同时写到redologbuffer与redo文件磁盘；写成功返回给客户成功。~~

我们也将更新的操作记录到change_buffer，降低跟磁盘的随机IO

**那么change_buffer里的数据怎么更新到磁盘?**

1.系统有后台线程定期执行合并操作，数据库正常关闭也会进行合并

2.当下次查询命中这个数据页的时候。会先从磁盘读取到数据页到内存，执行合并操作，然后再返回结果给用户，保证结果的正确

**是不是所有的数据都会用到changeBuffer?**

因为唯一索引每个插入或者更新都要判断是否违反唯一约束，所以每次更新都要从内存中找到这个记录对应的数据页，没有的话需要从磁盘中读出，反正都要从磁盘读数据页了，还不如直接更新内存/磁盘，那么也就意味着change buffer失去意义，所以唯一索引使用change buffer 的话反倒会降低效率，所以只有普通索引能使用到change buffer。

如果不用索引，那么bufferpool都不能用，直接操作磁盘！不然整个表放到内存，内存就爆了！！

**那么什么场景下是否使用changeBuffer?**

当写少读多的情况下：change buffer的利用率不高，因为可能刚一更新完可能就需要查询、就触发merge操作，change buffer没有起到减少随机IO，还多了一个维护change buffer的成本。

当写多读少的情况下：change buffer的提升效果较明显，可能很多条更新后，也没有一个查询线程来触发merge操作，可以大幅减少磁盘的随机IO。

# Mysql三大日志

# RedoLog

既然讲到了bufferpool，我们都知道bufferPool是一个内存空间，还有修改的数据都是先经过bufferpool的，那么假如数据在bufferPool，但是没有同步到磁盘，数据是不是就会丢失？ 也就是我们说的持久性！！

那么老师，我能不能在数据事务提交之前我就把修改的页面都刷新到磁盘行不行？肯定可以但是也会有问题

1.我只改了page页里的一条数据，但是会把整个page都会更新

2.随机IO比较慢

所以我们InnoDB中又有了redolog，作用就一个，就是我数据提交了就要保存，说到就要做到的，即使系统崩溃，我也能在重启后能把数据进行恢复。

我们不想因为改了page里的某条数据而导致需要马上刷新整个page，所以我们能不能把我们改动的数据放到另外一个地方，那么这个就是我们的redolog，redolog记录的就是我们的物理日志，比如我对page1的哪条数据做了哪些更改。

现在我们知道了磁盘上肯定有个这样的redolog，记录我们的每次操作，但是当初设计者为了解决磁盘太慢，所以有了bufferpool的概念，那么我们redolog也在磁盘，也很慢，那么能不能在bufferpoo里面开辟一个区间来放redolog，不用每次都往磁盘去操作！！！

我们可以通过启动参数innodb_log_buffer_size来指定log buffer的大小，在MySQL 5.7.21这个版本中，该启动参数的默认值为16MB。

所以redolog也有2部分

1.redo log buffer

咕泡出品

2.redo log file 日志文件（磁盘）

**那么redo log Buffer什么时候同步到磁盘?**

1.redo log buffer 使用量达到了参数`innndb_log_buffer_size`的一半时，会触发落盘。

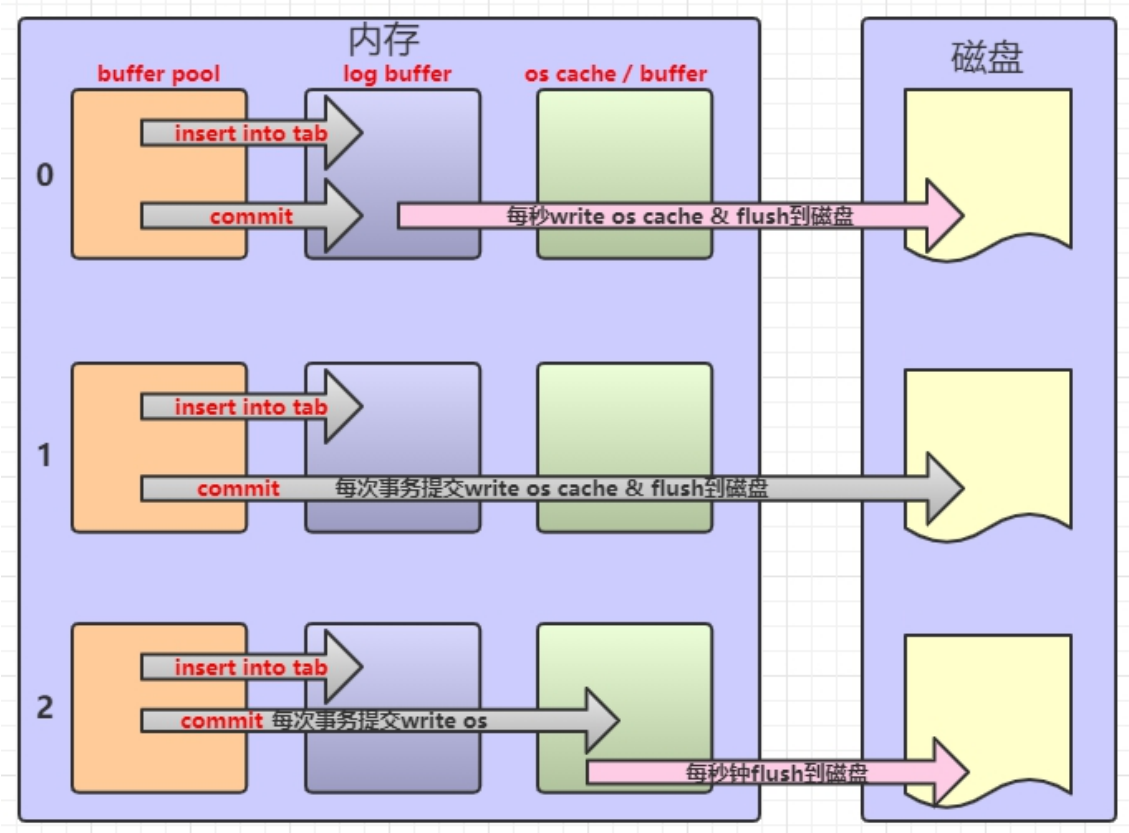
2.后台线程，每隔一段时间（1s）就会将redo log block刷新到磁盘文件中去。

3.Mysql正常关闭

4.事务提交时，可配 innodb_flush_log_at_trx_commit参数

该参数有几个选项：0、1、2

- 想要保证ACID四大特性推荐设置为1：表示当你commit时，MySQL必须将rodolog-buffer中的数据刷新进磁盘中。确保只要commit是成功的，磁盘上就得有对应的rodolog日志。这也是最安全的情况。
- 设置为0：每秒写一次日志并将其刷新到磁盘。
- 设置为2：表示当你commit时，将redolog-buffer中的数据刷新进OS Cache中，然后依托于操作系统每秒刷新一次的机制将数据同步到磁盘中，也存在丢失的风险。



# BinLog

刚才我们讲的redoLog属于InnoDB独有的，因为redolog的目的是为了事务里面的持久性，说到就要做到，所以它的实现是在InnoDB存储引擎去实现

但是BinLog就不只是InnoDB独有的，而是在我们的Service层实现的。

binlog是我们的二进制日志文件，记录了Mysql的所有的DML操作，我们可以通过binlog日志做数据恢复、增量备份、主从复制等！

**那么为什么要2个了？**

innodb利用wal技术进行数据恢复，write ahead logging技术依赖于物理日志进行数据恢复，binlog不是物理日志是逻辑日志，因此无法使用

redolog是循环写，写到末尾要回到开头继续写，这样的日志无法保留历史记录，无法进行数据复制。

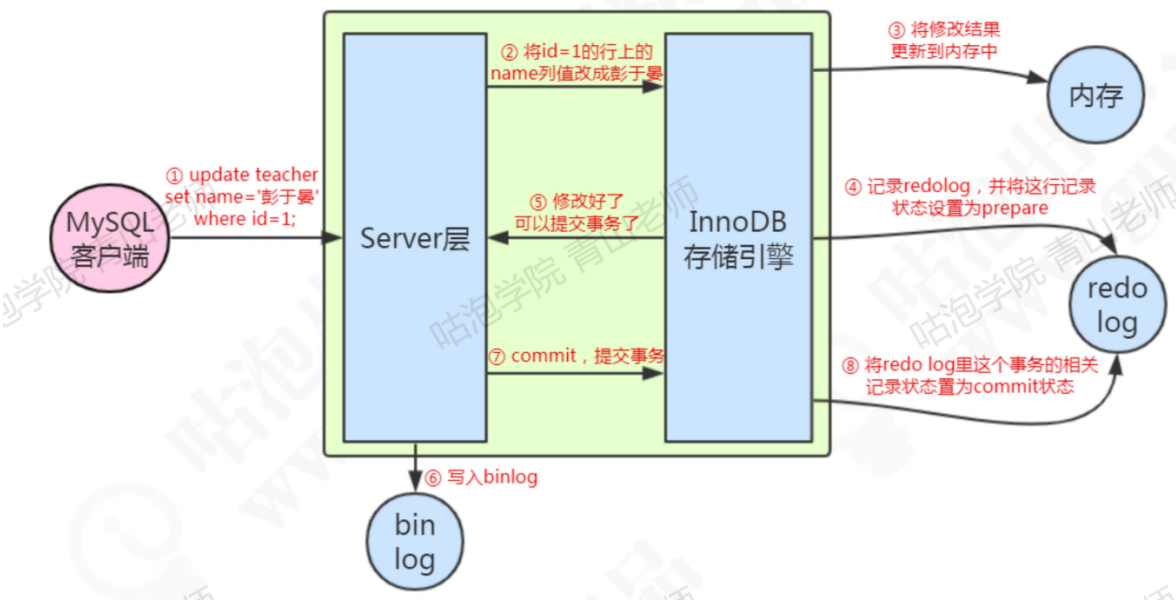
# 为什么需要RedoLog的二阶段提交

我们讲了2个log了，一个redo，一个binlog,一个是InnoDB存储引擎的，一个是server端的日志文件；这2个日志不是1个地方操作的，那么就不是原子性的；就会产生数据一致性问题；要么恢复的数据有问题，要么复制备份的数据会有问题。

假如我修改一条数据那么整体流程为

那么整体流程为：

咕泡出品




简单地来说，这里有两个写日志的操作，类似于分布式事务，不用两阶段提交，就不能保证都成功或者都失败。

在崩溃恢复时，判断事务是否需要提交：

1、binlog无记录，redolog无记录： 在redolog写之前crash，恢复操作：回滚事务

2、binlog无记录，redolog状态prepare：在binlog写完之前的crash，恢复操作：回滚事务

3、binlog有记录，redolog状态prepare： 在binlog写完提交事务之前的crash，恢复操作：提交事务

4、binlog有记录，redolog状态commit： 正常完成的事务，不需要恢复

# undoLog（回滚日志）

undo log（撤销日志或回滚日志）记录了事务发生之前的数据状态，分为insert undo log和update undo log。如果修改数据时出现异常，可以用undo log来实现回滚操作（保持原子性）。

写undo log 也会产生redo log

可以理解为undo log记录的是反向的操作，比如insert会记录delete，update会记录update原来的值，跟redolog记录在哪个物理页面做了什么操作不同，所以叫做逻辑格式的日志。

show global variables like '%undo%';

| 值 | 含义 |
| --- | --- |
| innodb_undo_directory | undo 文件的路径 |
| innodb_undo_log_truncate | 设置为1，即开启在线回收（收缩）undo log日志文件 |
| innodb_max_undo_log_size | 如果innodb_undo_log_truncate设置为1，超过这个大小的时候会触发truncate回收（收缩）动作，如果page大小是16KB，truncate后空间缩小到10M。默认1073741824字节=1G。 |
| innodb_undo_logs | 回滚段的数量， 默认128，这个参数已经过时。 |
| innodb_undo_tablespaces | 设置undo独立表空间个数，范围为0-95， 默认为0，0表示表示不开启独立undo表空间 且 undo日志存储在ibdata文件中。这个参数已经过时。 |

redo Log和undo Log与事务密切相关，统称为事务日志。

咕泡出品